



di.unito.it

DIPARTIMENTO
DI INFORMATICA

TLN

rappresentazioni vettoriali di
significato

Daniele Radicioni

outline

- vector space model
 - costruzione di uno spazio semantico multidimensionale
 - funzioni di distanza fra documenti
 - pesatura delle features
- metodo di Rocchio
- linear discriminant analysis

credits

- Jurafsky, D. (2000). Speech & language processing.
Pearson Education India.
 - Chapter 6: 6.8

vector space model

terminologia

- *documento* è un'unità di testo indicizzata nel sistema e disponibile per la ricerca.
- *collezione* è un insieme di documenti, utilizzati per soddisfare le richieste dell'utente.
- *termine* è l'elemento (dal singolo elemento lessicale a intere *phrases*) che occorre nella collezione.
- *interrogazione* è la richiesta dell'utente, espressa come insieme di termini.

term-document matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- each row represents a word in the vocabulary, and columns represent a document from some collection of documents
 - let us consider this matrix, describing **four plays by Shakespeare**, and **four sample terms**
 - each cell in the matrix says how many times a given term occurs in a given play

term-document matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- in this setting, a document can be described as a count vector:
 - *As You Like It* can be represented as the vector [1,114,36,20]

term-document matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- in this setting, a **document can be described as a count vector**:

- *As You Like It* can be represented as the vector [1,114,36,20]
- *Twelfth Night* can be represented as the vector [0,80,58,15]

- ...

vector space

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- a vector space is simply a collection of vectors, and is featured by their dimensions
- while in our example document vectors are of dimension 4, in real term-document matrices, the vectors representing each document would have dimensionality $|V|$, the vocabulary size (which corresponds to the number of rows, typically in the order of tens of thousands elements).
 - also the other dimension (number of documents in the collection) may be huge: think, e.g., to all web pages...

vector space

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- a vector space is simply a collection of vectors, and is featured by their dimensions
- while in our example document vectors are of dimension 4, in real term-document matrices, the vectors representing each document would have dimensionality $|V|$, the vocabulary size (which corresponds to the number of rows, typically in the order of tens of thousands elements).
 - also the other dimension (number of documents in the collection) may be huge: think, e.g., to all web pages...
- the first dimension for all vectors corresponds to the number of times the word *battle* occurs, and we can compare each dimension
 - which document should be returned if a user query asked for 'battle'?

$|V|$ -dimensional spaces

- to be able to plot vectors onto a 2-dimensional space, we restrict to considering 2 dimensions

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

from 4 dimensions

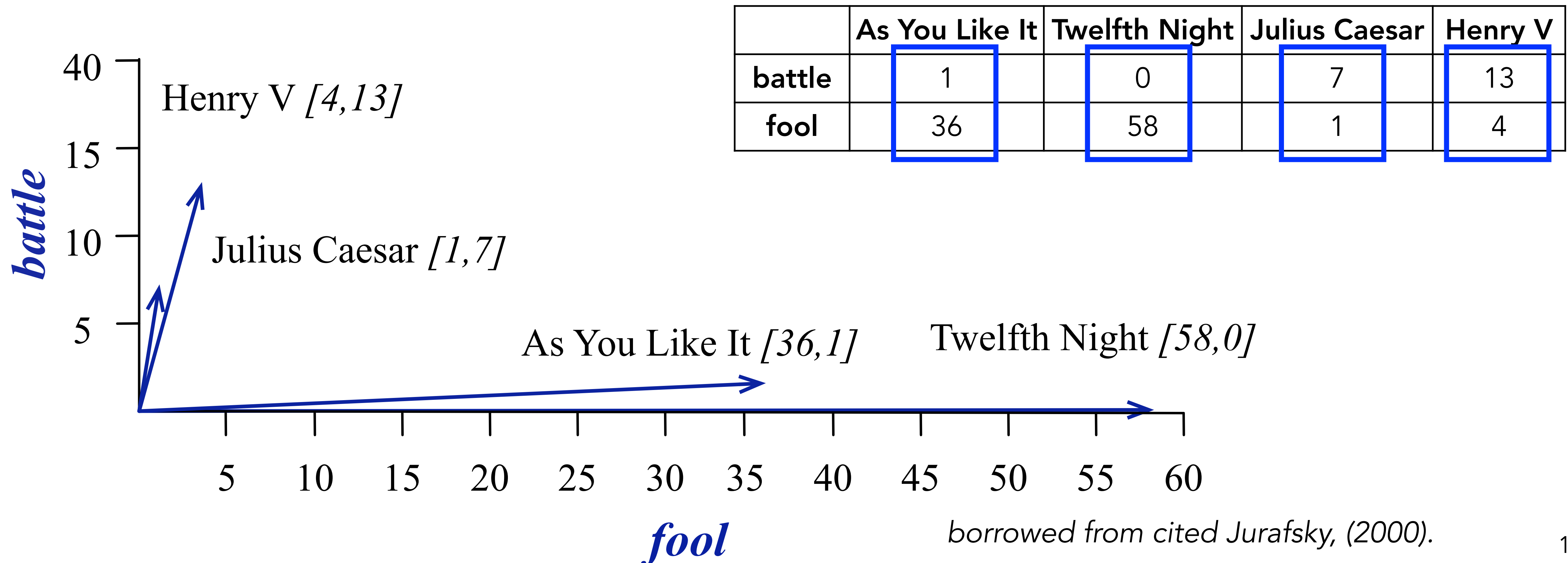


	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

to 2 dimensions

$|V|$ -dimensional spaces

- a vector for a document can be thought of as a point in $|V|$ -dimensional space;



information retrieval (IR): intuition

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
fool	36	58	1	4

- term-document matrices were developed as a means of **finding similar documents** for the IR task of finding the document that best matches a given query (another document)
 - two **documents are similar in so far that they contain similar words**
 - in our setting, this amounts to assuming that similar documents are reflected by similar vectorial representation
 - if we consider the term *battle*, it is clear that this is discriminative enough to distinguish comedies from tragedies...

IR task

- **information retrieval** is finding the document d from the D documents in some collection that best matches a query q .
 - queries are also represented as vectors, also of length $|V|$ (why do we consider queries of length $|V|$?)
- to solve this task we need to compare two vectors, to find how similar they are
 - implementation detail: efficient ways to store and manipulate these vectors are needed, by making use of the convenient fact that these vectors are **sparse**, i.e., mostly zeros

confrontare (vettori e) documenti

cosine similarity

- the most popular metrics is the cosine similarity
 - cosine is based on the **dot product** (or inner product) operator
- from linear algebra: $\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$
- dot product is a good metric because it **tends to be high when the vectors at stake have high values in the same dimensions**
 - vice versa, vectors having zero in different dimensions (orthogonal vectors) will have a dot product of 0, representing dissimilarity

cosine similarity

- the first version of dot product, $\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i$ tends to favor longer vectors
 - on the contrary, we are looking for a metrics that allows to compare documents regardless of their length, so to normalize our vectors we introduce **vector length**, which is defined as

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

cosine similarity

- we can modify the dot product to normalize for the vector length, by dividing the dot product by the lengths of the two vectors

- the **normalized dot product** is the cosine of the angle between the vectors at stake, since

$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos \theta \quad \Rightarrow \quad \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \cos \theta$$

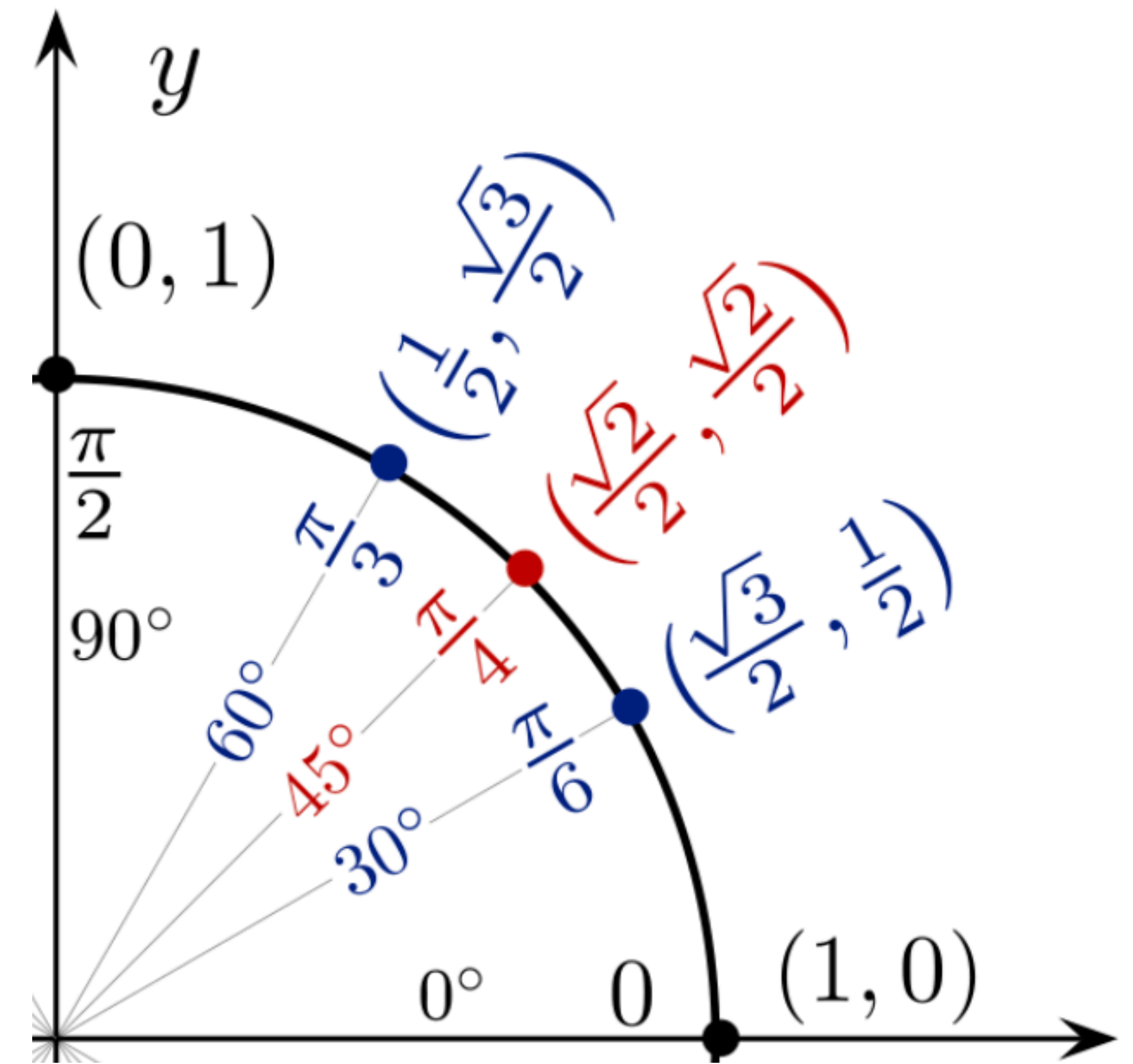
- the cosine similarity between two vectors $\mathbf{v}$ and $\mathbf{w}$ is computed as

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Daniele Radicioni - TLN

cosine similarity

- cosine ranges in $[-1,1]$, namely 1 for vectors with same direction, 0 for orthogonal vectors and -1 for vectors pointing in opposite directions
- however, since frequency values cannot be negative, in our setting cosine is bounded to the interval $[0,1]$



pesatura dei termini

pesatura dei termini

- finora abbiamo assunto che i *pesi dei termini* fossero determinati semplicemente con il conto della *frequenza dei vari termini nei documenti*.
- in realtà due fattori sono critici nell'attribuzione del giusto 'peso' ai termini:
 1. la *frequenza*;
 2. la frequenza in un documento ma la non-frequenza a livello dell'intera collezione

IDF: inverse document frequency

- l'idea è che i termini che sono presenti in pochi documenti sono utili per distinguere quei documenti dal resto della collezione (i termini che ricorrono in tutti i documenti non sono utili: e.g., *stop words*).
- pertanto l>IDF è definito per mezzo del rapporto N/n_i , dove N è il numero totale di documenti nella collezione, e n_i è il numero di documenti in cui il termine i occorre.

inverse document frequency

- quanti meno sono i documenti in cui un termine occorre, tanto più alto è il peso. il peso minimo è 1, corrispondente a termini che occorrono in tutti i documenti.
- dato l'elevato numero di documenti in varie collezioni, il rapporto è 'schiacciato' con una funzione logaritmica.

$$\text{idf}_i = \log \left(\frac{N}{n_i} \right)$$

inverse document frequency

- combinando *tf* e *idf* otteniamo il seguente schema:

$$w_{i,j} = tf_{i,j} \times idf_{i,j}$$

- utilizzando lo schema *tf-idf* il peso del termine *i* nel vettore del documento *j* è calcolato come il prodotto della sua frequenza totale in *j* per il logaritmo della sua *idf* nella collezione

$$\text{sim}(\vec{q}, \vec{d}) = \frac{\sum_{w \in q, d} tf_{w,q} tf_{w,d} (idf_w)^2}{\sqrt{\sum_{q_i \in q} (tf_{q_i,q} idf_{q_i})^2} \times \sqrt{\sum_{d_i \in d} (tf_{d_i,d} idf_{d_i})^2}}$$

il metodo di Rocchio

class profile

- Some linear classifiers consist of an *explicit profile* (or *prototypical* document) of the category.
- Learning a linear classifier is often preceded by local *term space reduction*; in this case, a profile of c_i is a weighted list of the terms whose presence or absence is most useful for discriminating c_i .

centroid

- Let D be the **set of documents** and T be the **dictionary**, i.e. the set of all different terms occurring in D .

$$T = \{t_1, \dots, t_m\}$$

- then the **absolute frequency** of term $t \in T$ in document $d \in D$ is given by $tf(d, t)$.
- We denote the term vectors

$$\vec{t}_d = (tf(d, t_1), \dots, tf(d, t_m))$$

centroid

- il *centroide* di un insieme di X vettori di termini è definito come

$$\vec{t}_X := \frac{1}{|X|} \sum_{\vec{t}_d \in X} \vec{t}_d$$

Rocchio method: intuition

- A classifier built by means of the Rocchio method rewards
 - the closeness of a test document to the centroid of the positive training examples, and
 - its distance from the centroid of the negative training examples.

the Rocchio method

$$\vec{c}_i = \langle f_{1i}, \dots, f_{|\mathcal{T}|i} \rangle$$

- for each class c_i (with i varying over classes) we compute the weight of the k -th feature f as

$$f_{ki} = \beta \cdot \sum_{d_j \in POS_i} \frac{w_{kj}}{|POS_i|} - \gamma \cdot \sum_{d_j \in NEG_i} \frac{w_{kj}}{|NEG_i|}$$

$$POS_i = \{d_j \in Tr | \Phi(d_j, c_i) = T\}$$

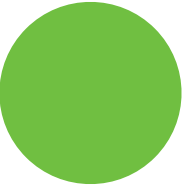
$$NEG_i = \{d_j \in Tr | \Phi(d_j, c_i) = F\}$$

- β and γ are **control parameters** that allow setting the relative importance of positive and negative examples
 - if β is set to 1 and γ to 0, the profile of c is the centroid of its positive training examples.

raffinamento: i near positives

$$f_{ki} = \beta \cdot \sum_{d_j \in POS_i} \frac{w_{kj}}{|POS_i|} - \gamma \cdot \sum_{d_j \in NPOS_i} \frac{w_{kj}}{|NPOS_i|}$$

- **Open issue**: whether the set NEG_i should be considered in its entirety, or whether a well- chosen sample of it, such as the set $NPOS_i$ of near-positives (defined as "*the most positive among the negative training examples*").
 - Near-positives are the most difficult documents to tell apart from the positives.

Linear discriminant analysis (LDA)

linear discriminant analysis

- in text classification, a document can be converted into a numerical representation through the Vector Space Model (VSM)
 - given a vocabulary $V = \{t_1, \dots, t_p\}$, each document d_i is represented by a vector $x_i = (w_{i1}, w_{i2}, \dots, w_{ip}) \in \mathbb{R}^p$, where w_{ij} measures the importance of term t_j in document d_i .
- depending on the chosen representation, these weights may be binary values, term frequencies, or TF-IDF scores.
 - since each document contains only a small subset of the whole vocabulary, these vectors are usually sparse.

linear discriminant analysis

- once documents have been embedded in this vector space, the classification problem becomes a standard supervised learning problem: from a set of training pairs (x_i, y_i) , where $y_i \in \{0,1\}$, we want to learn a rule that assigns a new document to one of the two classes.

linear discriminant analysis

- linear Discriminant Analysis (LDA) approaches this problem from a probabilistic perspective: it assumes that the vectors belonging to each class are generated from a multivariate Gaussian distribution.
 - In the binary case, the model assumes
$$x \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma), \quad x \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$$
 - that is, the two classes have different mean vectors μ_0 and μ_1 , but share the same covariance matrix Σ .
- the prior probabilities of the classes are denoted by
$$P(y = 0) = \pi_0, \quad P(y = 1) = \pi_1 .$$

linear discriminant analysis

- The mean vector of each class describes the average position of that class in the term space, while the common covariance matrix describes how the components vary jointly around their class means.

- From these assumptions, LDA derives for each class a discriminant function

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k, \quad k \in \{0,1\}.$$

- a document x is assigned to the class with the largest discriminant score. in the binary case, this leads to a linear decision rule of the form

$$w^T x + b \geq 0,$$

- so LDA defines a linear separating hyperplane in the document space. For this reason, although the data come from text, the final classifier is linear in the vector representation produced by the VSM.

linear discriminant analysis

- in practice, the unknown parameters μ_0 , μ_1 , Σ , π_0 , and π_1 are estimated from the training set:
 - class priors are estimated from class frequencies,
 - class means from average document vectors, and
 - the common covariance matrix from a pooled estimate over both classes.
- once these quantities are known, the classifier can be applied to any new sparse document vector.
- text representations are often very high-dimensional and sparse, so covariance estimation may become difficult when the vocabulary is large.

how to build covariance matrix

- in the Vector Space Model, each document is represented by a column vector

$$x = (x_1, x_2, \dots, x_p)^T$$

- where each component corresponds to a feature, for example the weight of a term in the vocabulary.
- when we look at many documents from the same class, these vectors vary from one document to another.
- the covariance matrix is the object that describes this variability jointly across all features.

covariance matrix

- for a random vector $X = (X_1, \dots, X_p)^T$, the covariance matrix is defined as $\Sigma = \text{Cov}(X)$ with entries $\Sigma_{ij} = \text{Cov}(X_i, X_j)$.
- thus, Σ is a $p \times p$ matrix whose:
 - diagonal entries Σ_{ii} are the variances of the individual features,
 - off-diagonal entries Σ_{ij} measure how pairs of features vary together.

covariance matrix

- in text classification the covariance matrix captures not only how much each term weight varies across documents, but also whether some terms tend to appear together or in opposition.
 - if $\text{Cov}(X_i, X_j) > 0$, the two features tend to increase together,
 - if $\text{Cov}(X_i, X_j) < 0$, when one increases the other tends to decrease,
 - if $\text{Cov}(X_i, X_j) \approx 0$, they show little linear co-variation.

distribuzioni

- In LDA, we assume that for each class the document vectors follow a multivariate Gaussian distribution, and that all classes share the same covariance matrix,
$$X \mid Y = 0 \sim \mathcal{N}(\mu_0, \Sigma), \quad X \mid Y = 1 \sim \mathcal{N}(\mu_1, \Sigma).$$
- the mean vector μ_k tells us where class k is centered,
- the covariance matrix Σ tells us the shape and spread of the cloud of points around that center.

covariance matrix

- Σ is unknown and must be estimated from the training set.
- suppose we have n document vectors $x_1, \dots, x_n$ from one class, and let

$$\bar{x} = \frac{1}{n} \sum_{r=1}^n x_r \text{ be the sample mean vector.}$$

- the sample covariance matrix is $S = \frac{1}{n-1} \sum_{r=1}^n (x_r - \bar{x})(x_r - \bar{x})^T$;
 - each term $(x_r - \bar{x})(x_r - \bar{x})^T$ is an outer product, producing a $p \times p$ matrix. summing these and dividing by $n-1$ gives the covariance estimate.

covariance matrix

- in LDA with two classes, we can estimate a pooled covariance matrix:

$$\hat{\Sigma} = \frac{\sum_{x_i \in D_0} (x_i - \hat{\mu}_0)(x_i - \hat{\mu}_0)^T + \sum_{x_i \in D_1} (x_i - \hat{\mu}_1)(x_i - \hat{\mu}_1)^T}{n_0 + n_1 - 2}.$$

- this reflects the LDA assumption that both classes share the same covariance structure; n_0 is the number of support items in class 0; same holds for n_1 for class 1.

estimate LDA parameters all in one slide (!)

- given $D_0 = \{x_i : y_i = 0\}$, and $D_1 = \{x_i : y_i = 1\}$ compute:
- the class priors $\hat{\pi}_0 = \frac{n_0}{n}$, $\hat{\pi}_1 = \frac{n_1}{n}$;
- the class mean vectors $\hat{\mu}_0 = \frac{1}{n_0} \sum_{x_i \in D_0} x_i$, $\hat{\mu}_1 = \frac{1}{n_1} \sum_{x_i \in D_1} x_i$
- the pooled covariance matrix

$$\hat{\Sigma} = \frac{\sum_{x_i \in D_0} (x_i - \hat{\mu}_0)(x_i - \hat{\mu}_0)^T + \sum_{x_i \in D_1} (x_i - \hat{\mu}_1)(x_i - \hat{\mu}_1)^T}{n_0 + n_1 - 2}$$

the categorization rule

- for each class k , we compute the LDA discriminant score

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

- and assign x to the class with largest score.
 - $x^T \Sigma^{-1} \mu_k$ measures the **alignment between the document x and the class mean μ_k** , taking the covariance structure into account.
 - $-\frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k$ is a **correction term that depends only on the class center**; it prevents classes with large mean vectors from being favored automatically.
 - $\log \pi_k$ incorporates the prior probability of class k , so **more frequent classes are slightly advantaged**.

LDA esempio

on the role of covariance information

- suppose each document is represented by two features, $x = (x_1, x_2)$ and the two class means are $\mu_0 = (1,1)$; $\mu_1 = (3,2)$.
 - the raw mean difference is $\mu_1 - \mu_0 = (2,1)$.
- if we ignored covariance, we might think feature x_1 is twice as important as feature x_2 .

on the role of covariance information

- for two classes, the rule can be rewritten as $\delta_1(x) - \delta_0(x) = w^T x + b$
- suppose the common covariance matrix is $\Sigma = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$; then

$$\Sigma^{-1} = \begin{pmatrix} 1/4 & 0 \\ 0 & 1 \end{pmatrix}.$$

- therefore, $w = \Sigma^{-1}(\mu_1 - \mu_0) = \begin{pmatrix} 1/4 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1/2 \\ 1 \end{pmatrix}$.
- after covariance adjustment, feature x_2 becomes more important than feature x_1 , even though the raw mean difference was larger on x_1 .
 - because x_1 has much larger variance within the classes, so it is less reliable for discrimination

